

Embedded Software Project: STM32 Peripheral Drivers (I2C, SPI, USART, GPIO)

1. Project Objective

In this project, you will work with a **bare-metal STM32F446RE/STM32F401RE driver library** that provides ready-made APIs for:

- **GPIO**
- **USART**
- **I2C**
- **SPI**

Your goal is to:

- Study how each driver is implemented.
- Understand how the example application code uses the driver APIs.
- Adapt the code to your own board setup (if different from the reference).
- Implement and test all major APIs in practical examples.
- Record a short video demonstrating the working of selected APIs.

This project will strengthen your understanding of **embedded driver development, peripheral configuration, and application-level usage of drivers**.

2. Project Repository

You will use the following GitHub repository as the base code package:

- **Driver repository:**
`https://github.com/ghostcoder911/STM32F446RE\_Drivers.git`

Clone this repository and study its structure carefully. The repository contains:

- **Driver source files** (GPIO, USART, I2C, SPI).
 - **MCU-specific header** for STM32F446RE/STM32F401RE .
 - **Example code** that demonstrates how to use the driver APIs (look for `main.c` and example tasks or test folders).
-

3. What You Need to Do

Step 1: Deep Dive into the Drivers

For each peripheral (GPIO, USART, I2C, SPI), you must:

1. Open the corresponding driver files (.c and .h).
2. Read and annotate:
 - Register-based configuration (clock enable, mode, speed, pull-up/down, etc.).
 - Function signatures and parameters.
 - How the driver abstracts hardware into easy-to-use APIs.
3. Write a brief **1–2 page explanation per driver** covering:
 - How initialization is done.
 - How data transfer (or I/O) is performed.
 - Any design choices (e.g., blocking vs. interrupt-based, polling-based implementation).

Step 2: Study Example / Application Code

The repository may already contain example code such as:

- Blinking an LED using GPIO.
- Transmitting and receiving data over USART.
- Reading from or writing to an I2C sensor.
- Sending data over SPI (e.g., to an LCD or sensor).

You must:

- Trace how each driver API is called from the example code.
- Note down the **pin configuration**, **clock setup**, and **any board-specific macros**.
- Identify what changes are needed if your board layout differs (e.g., different LED pin, different UART pins).

Step 3: Porting / Modifying for Your Board

If your students' boards differ from the reference board (e.g., different STM32F446/401RE-based dev board like Nucleo-F446RE/401RE), they must:

1. Map the **physical pins** used in the example to the actual pins on their board.
2. Update:
 - GPIO port/pin definitions.
 - USART/I2C/SPI instance numbers (e.g., USART2 vs USART1).

- Clock configuration (if using different APB buses).

3. Ensure the code builds and runs without relying on the original example board layout.

Students may need to:

- Edit header files (e.g., `AppConfig.h`, `BoardConfig.h`).
- Define new macros or constants for their board.
- Optionally add their own `main.c` or example tasks.

Step 4: Implement New Examples (Mandatory)

You must **create and demonstrate** at least **one additional example application** for **each** of the following using the provided APIs:

Peripheral	Example Task
GPIO	Read a button and toggle an LED; or generate a PWM-like LED fade using software timing.
USART	Echo received characters; or print a structured message (e.g., “Button pressed”) over UART to a serial terminal.
I2C	Read temperature from an I2C sensor (e.g., LM75 or similar) and print it over UART.
SPI	Drive an SPI-based device such as an OLED/LCD, or just send a test pattern over SPI and capture it with a logic analyzer.

You are allowed to pick **real hardware** (sensors, LEDs, push buttons) or **simulate with LEDs/switches** if needed.

Step 5: Video Demonstration

Each student (or group) must record a **short video (2–4 minutes)** that shows:

1. The **hardware setup** (board, cables, peripherals connected).
2. The **code repository and key files** (you can briefly show the driver and example code on screen).
3. The **working of at least 3 APIs**, for example:
 - LED blinking via GPIO.
 - USART echo or sensor data printing.
 - I2C sensor reading or SPI data transfer.
4. **Optional but encouraging:** Show a small “problem” (e.g., wrong pin selected) and then how you fixed it.

The video can be uploaded as:

- A file on Google Drive / YouTube (unlisted) / OneDrive, etc., with a link shared in the project report.
-

4. Project Deliverables

Each student (or team) must submit:

1. Driver Analysis Document (PDF)

- 1–2 pages per driver (GPIO, USART, I2C, SPI).
- Explain how each driver works, how it hides hardware complexity, and what APIs it offers.

2. Modified Code Package

- A ZIP folder or GitHub link containing:
 - Your modified driver or configuration files.
 - Your example/application code (e.g., `main.c`, custom tasks).
 - Any new macros or board-specific changes.

3. Video Link

- A clear link to the working-demo video.
- The description of each demonstration must be written below the video link (e.g., “0:15–0:45: GPIO LED toggle using GPIO driver API”).

4. Short Reflection (Optional)

- 1–2 paragraphs on:
 - What was easy / hard in understanding the driver code.
 - How much you had to modify for your board.
 - What you learned about embedded driver design.
-

5. Learning Outcomes

By completing this project, you will:

- Understand how to **write and use peripheral drivers** in bare-metal embedded systems.
 - Learn to **map hardware-specific details** (pins, clocks, registers) into reusable APIs.
 - Gain experience in **porting drivers** across similar boards.
 - Practice **debugging and testing** communication peripherals (I2C, SPI, USART) with real hardware or LEDs.
 - Improve your ability to **document drivers and example usage** clearly.
-

6. Tips for Students

- Start with **GPIO** first (simplest) before moving to USART, I2C, and SPI.
 - Use **ST's reference manual** and **STM32F446RE/401RE datasheet** to cross-check register names and bit-fields.
 - Keep your modifications **clean and modular** (create new header files for board-specific settings).
 - If you get stuck, **inspect the example code** and compare it with the driver header to see how each function is meant to be used.
-